



Marco Teórico

Bayes Web Diagnosis: Razonamiento Probabilístico y Redes Bayesianas

Basado en el Capítulo 13: *Probabilistic Reasoning* de **Artificial Intelligence: A Modern Approach** (Russell & Norvig).

1. Razonamiento bajo incertidumbre

En el diagnóstico de sistemas reales, rara vez se dispone de información completa. Un agente inteligente debe tomar decisiones a partir de evidencia parcial, ambigua o ruidosa. El **razonamiento probabilístico** es el enfoque formal que permite cuantificar esa incertidumbre y actualizar las creencias del agente a medida que se observan nuevos datos.

El fundamento matemático es la **regla de Bayes**, que establece cómo revisar una probabilidad a priori al conocer nueva evidencia:

$$P(\text{causa} \mid \text{evidencia}) = \frac{P(\text{evidencia} \mid \text{causa}) \cdot P(\text{causa})}{P(\text{evidencia})}$$

En el proyecto, esta idea se aplica directamente: dado que se observa `Error500 = sí` y `LatenciaAlta = sí`, ¿cuán probable es que `BaseDatosCaida = sí`? El sistema no responde con una certeza absoluta, sino con una distribución de probabilidad sobre las posibles causas.

2. Redes Bayesianas

Una **red bayesiana** (también llamada red de creencia o grafo acíclico dirigido probabilístico) es una representación compacta de una distribución de probabilidad conjunta sobre un conjunto de variables aleatorias.

2.1 Definición formal

Formalmente, una red bayesiana es un par (G, Θ) donde:

- **G** es un grafo acíclico dirigido (DAG) cuyos nodos representan variables aleatorias $X_1, X_2, \dots, X_n$.
- **Θ** es el conjunto de tablas de probabilidad condicional, donde cada tabla $\theta_{X_i|Pa(X_i)}$ especifica $P(X_i \mid Pa(X_i))$ para cada nodo X_i y sus padres $Pa(X_i)$.

La propiedad fundamental es que cada variable es **condicionalmente independiente** de sus no-descendientes dado el estado de sus padres. Esto permite expresar la distribución conjunta completa como un producto de distribuciones locales:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid Pa(X_i))$$

2.2 Estructura en el proyecto

En **Bayes Web Diagnosis**, la red tiene 10 variables booleanas organizadas en dos niveles causales:

Nodos causa (sin padres — variables independientes):

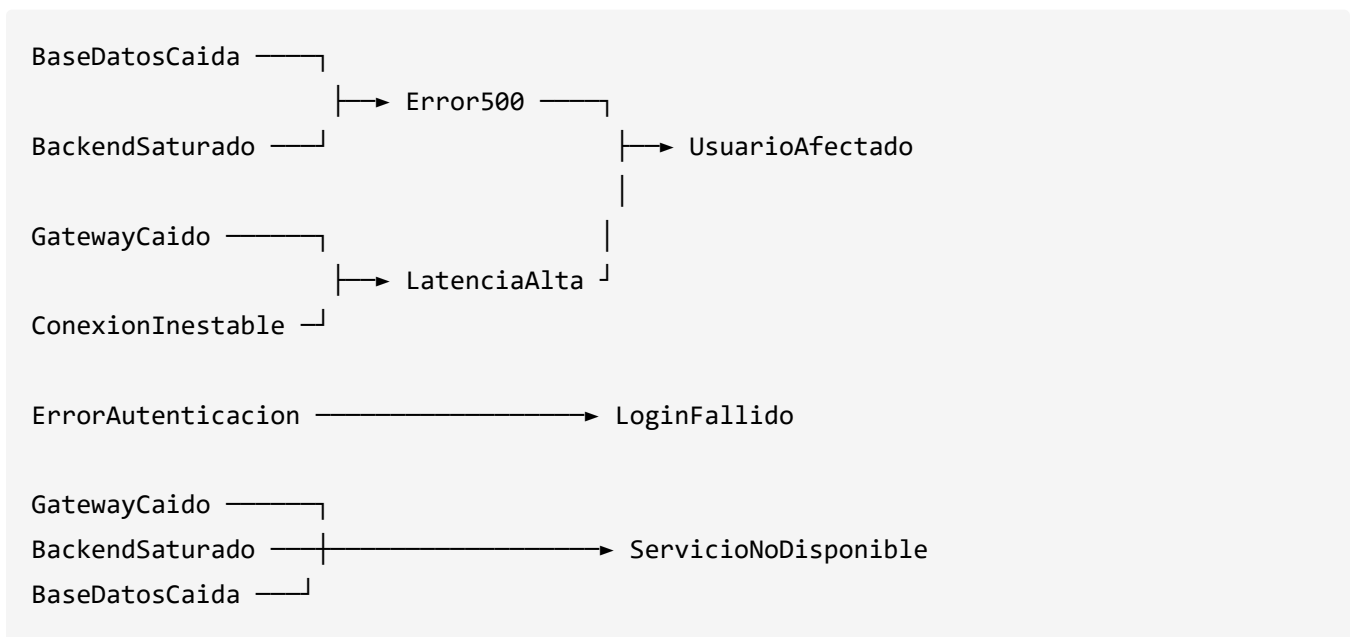
Variable	P(variable = True)	Significado
BaseDatosCaida	0.05	La base de datos presenta falla
BackendSaturado	0.10	El backend está sobrecargado
GatewayCaido	0.04	El API Gateway no responde
ConexionInestable	0.12	La red presenta inestabilidad
ErrorAutenticacion	0.08	El servicio de autenticación falla

Nodos síntoma (con padres — variables dependientes):

Variable	Padres	Significado
Error500	BaseDatosCaida, BackendSaturado	Error HTTP interno del

Variable	Padres	Significado
		servidor
LatenciaAlta	GatewayCaído, ConexionInestable	Respuestas lentas del sistema
LoginFallido	ErrorAutenticacion	Fallo al iniciar sesión
ServicioNoDisponible	GatewayCaído, BackendSaturado, BaseDatosCaida	Servicio inaccesible
UsuarioAfectado	Error500, LatenciaAlta	Impacto percibido por el usuario

La estructura causal del grafo es:



Esta estructura está codificada en `model.py` mediante la clase `Node`, donde cada nodo declara explícitamente su lista de padres. La clase `BayesianNetwork` en `bayesian_network.py` almacena todos los nodos en un diccionario y mantiene el orden topológico.

3. Tablas de Probabilidad Condicional (CPT)

La **tabla de probabilidad condicional** (CPT, *Conditional Probability Table*) de un nodo X_i especifica la probabilidad de que X_i sea verdadero para cada combinación posible de valores de sus padres.

3.1 Estructura general

Para un nodo con k padres booleanos, la CPT tiene 2^k filas. Cada fila corresponde a una combinación de valores de los padres y contiene $P(X_i = True \mid Pa(X_i))$. La probabilidad complementaria $P(X_i = False \mid Pa(X_i)) = 1 - P(X_i = True \mid Pa(X_i))$ se obtiene implícitamente.

3.2 Ejemplo: nodo `Error500`

El nodo `Error500` tiene dos padres: `BaseDatosCaída` y `BackendSaturado`.

BaseDatosCaída	BackendSaturado	P(Error500 = True)	Interpretación
True	True	0.95	Ambas causas activas: casi certeza de error
True	False	0.85	BD caída es causa fuerte del error 500
False	True	0.70	Backend saturado también genera errores 500
False	False	0.05	Ruido de fondo: causas externas no modeladas

En el código (`model.py`), esto se representa como:

```
nodo_error500 = Node(
    name="Error500",
    parents=["BaseDatosCaida", "BackendSaturado"],
    cpt={
        (True, True): 0.95,
        (True, False): 0.85,
        (False, True): 0.70,
        (False, False): 0.05,
    }
)
```

La clase `Node` en `node.py` valida automáticamente que el número de filas de la CPT sea exactamente 2^k , donde k es el número de padres, y que todas las probabilidades estén en el rango $[0, 1]$.

3.3 Consulta de probabilidad local

El método `get_probability` de la clase `Node` recupera la probabilidad local del nodo dado el estado actual de la evidencia:

```
def get_probability(self, value: bool, evidence: Dict[str, bool]) -> float:
    parent_values = tuple(evidence[parent] for parent in self.parents)
    p_true = self.cpt[parent_values]
    return p_true if value else 1.0 - p_true
```

Esta operación corresponde directamente a evaluar $P(X_i = valor \mid Pa(X_i) = pa_i)$, el factor local que el algoritmo de inferencia por enumeración multiplica en cada paso.

4. Independencia Condicional

La eficiencia de las redes bayesianas proviene de su capacidad de explotar la **independencia condicional** entre variables.

4.1 Definición

Dos variables X e Y son **condicionalmente independientes** dado Z si:

$$P(X | Y, Z) = P(X | Z)$$

Es decir, conocer Y no aporta información adicional sobre X cuando ya se conoce Z .

4.2 La propiedad de Markov local

En una red bayesiana, cada nodo es condicionalmente independiente de todos sus **no-descendientes** dado el estado de sus **padres**. Esta propiedad, conocida como la **condición de Markov local**, es la que permite factorizar la distribución conjunta en el producto de distribuciones locales presentado en la Sección 2.1.

4.3 Aplicación en el proyecto

En la red de `BayesWebDiagnosis`, la independencia condicional permite razonar de forma modular:

- `UsuarioAfectado` es condicionalmente independiente de `BaseDatosCaida`, `BackendSaturado`, `GatewayCaido`, `ConexionInestable` y `ErrorAutenticacion` dado el estado de `Error500` y `LatenciaAlta` (sus padres directos). Esto significa que la causa raíz de los problemas no afecta directamente al usuario: solo importa si hay errores o lentitud.
- `LoginFallido` es condicionalmente independiente de todas las demás variables dado `ErrorAutenticacion`. Un fallo de login no está relacionado con la latencia ni con los errores 500, a menos que ambos se expliquen por una causa común.

Sin estas independencias, calcular $P(X_1, X_2, \dots, X_{10})$ requeriría una tabla con $2^{10} = 1024$ entradas. Con la estructura de la red bayesiana, el modelo queda completamente especificado por las CPTs de cada nodo, cuyo total de parámetros es mucho menor.

5. Inferencia por Enumeración

El **algoritmo de inferencia por enumeración** (*Enumeration-Ask*) calcula la distribución de

probabilidad posterior de una variable de consulta Q dado un conjunto de evidencias e , sumando sobre todas las combinaciones posibles de las variables ocultas (no observadas y no consultadas).

5.1 Algoritmo formal

El algoritmo se basa en la siguiente expresión, derivada directamente de la regla de Bayes y la factorización de la distribución conjunta:

$$P(Q \mid e) = \alpha \sum_h P(Q, e, h)$$

donde h recorre todos los posibles valores de las variables ocultas H , y α es la constante de normalización.

La distribución conjunta se evalúa usando la factorización de la red:

$$P(Q, e, h) = \prod_{i=1}^n P(X_i \mid Pa(X_i))$$

El algoritmo procede de forma recursiva, siguiendo el orden topológico de las variables:

```
función ENUMERATION-ASK(Q, e, bn):
    para cada valor q de Q:
        distribución[q] ← ENUMERATE-ALL(Variables(bn), e ∪ {Q=q}, bn)
    retornar NORMALIZAR(distribución)

función ENUMERATE-ALL(vars, e, bn):
    si vars está vacío: retornar 1.0
    Y ← primera variable de vars
    si Y está en e:
        retornar P(Y=e[Y] | Pa(Y)) × ENUMERATE-ALL(resto(vars), e, bn)
    sino:
        retornar Σ_y P(Y=y | Pa(Y)) × ENUMERATE-ALL(resto(vars), e ∪ {Y=y}, bn)
```

5.2 Implementación en `inference.py`

El código del proyecto implementa este algoritmo de manera fiel al pseudocódigo del Capítulo

13. A continuación se muestra la correspondencia directa:

enumeration_ask — calcula la distribución posterior para cada posible valor de la variable de consulta:

```
def enumeration_ask(query, evidence, network):
    distribution = {}
    for value in (True, False):
        extended_evidence = dict(evidence)
        extended_evidence[query] = value
        distribution[value] = enumerate_all(
            network.get_variables(), extended_evidence, network
        )
    return normalize(distribution)
```

enumerate_all — recorre recursivamente las variables en orden topológico:

```
def enumerate_all(variables, evidence, network):
    if not variables:
        return 1.0  # caso base

    first, rest = variables[0], variables[1:]

    if first in evidence:  # variable observada
        probability = network.probability(first, evidence[first], evidence)
        return probability * enumerate_all(rest, evidence, network)

    total = 0.0  # variable oculta
    for value in (True, False):
        extended_evidence = dict(evidence)
        extended_evidence[first] = value
        probability = network.probability(first, value, extended_evidence)
        total += probability * enumerate_all(rest, extended_evidence, network)
    return total
```


5.3 Correspondencia con el libro

Concepto del Capítulo 13	Implementación en el proyecto
<code>ENUMERATION-ASK(X, e, bn)</code>	función <code>enumeration_ask</code> en <code>inference.py</code>
<code>ENUMERATE-ALL(vars, e)</code>	función <code>enumerate_all</code> en <code>inference.py</code>
$P(y \mid pa(Y))$ — factor local	método <code>get_probability</code> en <code>node.py</code>
<code>NORMALIZE(Q)</code>	función <code>normalize</code> en <code>inference.py</code>
Orden topológico de variables	atributo <code>order</code> en <code>bayesian_network.py</code>
CPT del nodo	atributo <code>cpt</code> en la clase <code>Node</code>

5.4 Normalización

Después de calcular los valores proporcionales para $Q = True$ y $Q = False$, el algoritmo aplica la normalización:

$$P(Q = True \mid e) = \frac{\hat{P}(Q = True)}{\hat{P}(Q = True) + \hat{P}(Q = False)}$$

En el código:

```
def normalize(distribution):
    total = sum(distribution.values())
    return {key: value / total for key, value in distribution.items()}
```

Esto garantiza que la distribución resultante sea válida, es decir, que sus valores sumen exactamente 1.

5.5 Complejidad

El algoritmo de inferencia por enumeración tiene complejidad **exponencial** en el número de variables ocultas: $O(n \cdot 2^n)$ en el peor caso, donde n es el total de variables. Sin embargo, para redes de tamaño moderado como la de este proyecto (10 variables), esta complejidad es completamente manejable. Los tiempos de ejecución medidos en `scenarios.py` confirman

que cada consulta se resuelve en fracciones de milisegundo.

6. Validación teórica del algoritmo implementado

Para verificar que la implementación corresponde fielmente al algoritmo del Capítulo 13, se pueden aplicar las siguientes comprobaciones:

6.1 Coherencia con probabilidades a priori

Cuando no se proporciona ninguna evidencia, `enumeration_ask` debe devolver la probabilidad marginal a priori de la variable consultada. Por ejemplo:

```
P(BaseDatosCaida | sin evidencia) ≈ 0.05
```

Esto se verifica en `scenarios.py` mediante la función `compare_prior_vs_posterior`, que compara explícitamente las probabilidades a priori y a posteriori para cada escenario.

6.2 Monotonicidad de la evidencia

Al agregar evidencia consistente con una causa, su probabilidad debe aumentar. En el Escenario 1:

- A priori: $P(\text{BaseDatosCaida}) = 0.05$
- Con `Error500=True, LatenciaAlta=True`: la probabilidad sube considerablemente.

Esto se confirma en la salida del análisis de sensibilidad, donde se muestran todas las variaciones de evidencia para cada escenario.

6.3 Validez de la distribución

La suma de probabilidades siempre debe ser 1:

$$P(Q = \text{True} \mid e) + P(Q = \text{False} \mid e) = 1.0$$

La función `normalize` en `inference.py` garantiza esta propiedad, y lanza `ValueError` si la distribución sin normalizar suma 0 (caso imposible con CPTs correctamente definidas).

6.4 Validación estructural

La clase `BayesianNetwork` implementa el método `validate_network`, que verifica:

- Que todos los padres declarados existan en la red.
- Que el orden topológico sea correcto (los padres siempre aparecen antes que sus hijos).

Esto asegura que el algoritmo de inferencia pueda recorrer las variables en orden topológico sin encontrar dependencias no resueltas.

7. Conclusión

El proyecto **Bayes Web Diagnosis** aplica de manera directa y completa los conceptos del Capítulo 13 de Russell & Norvig:

- La **red bayesiana** está representada mediante grafos dirigidos acíclicos con variables booleanas (`node.py`, `bayesian_network.py`, `model.py`).
- Las **tablas de probabilidad condicional** definen las relaciones causales entre fallas y síntomas (`model.py`).
- La **independencia condicional** está implícita en la estructura de la red y permite un modelo compacto con parámetros bien justificados.
- El **algoritmo de inferencia por enumeración** está implementado de forma fiel al pseudocódigo del libro, con normalización explícita de los resultados (`inference.py`).

La combinación de estos elementos produce un sistema capaz de responder preguntas del tipo $P(\text{causa} \mid \text{síntomas observados})$, que es exactamente el problema planteado: diagnosticar probabilísticamente la causa más probable de una falla en una aplicación web distribuida.